

Récupération des poids F et J depuis ProfileMLP

Comment `extract_effective_FJ` retrouve un modèle de Potts à partir d'un réseau qui n'a jamais vu que des scores scalaires

Institut de la Vision — Directed Evolution Loop

Contents

1	Le problème	2
2	Idée générale : sonder le réseau plutôt que lire ses poids	2
3	Récupérer F : balayage mono-mutant	2
4	Récupérer J : balayage double-mutant	3
5	Le code (<code>extract_effective_FJ</code>)	3
6	Vérifier la récupération	4
7	Limites	5
8	Résumé	5

1 Le problème

ProfileMLP (`ProfileMLP_recovery_nnx.ipynb`) prend en entrée un vecteur one-hot mono-site $x \in \mathbb{R}^{L \times A}$ ($L = 7$ positions, $A = 20$ acides aminés) et ressort **un seul score scalaire** $\mathcal{M}(x)$ — le log-ratio d'enrichissement prédit pour la séquence. C'est la seule chose qu'une vraie expérience NGS observe : jamais les 140 poids de profil ni les $21 \times 20 \times 20 = 8400$ couplages de paires directement.

Le simulateur (`protocol.compute_score`), lui, calcule ce score à partir d'un modèle de Potts complet :

$$\text{score}(s) = \sum_{p=0}^{L-1} F(s_p, p) + \sum_{0 \leq i < j \leq L-1} J(i, j, s_i, s_j),$$

où $F \in \mathbb{R}^{A \times L}$ sont les poids de profil (effet additif de chaque acide aminé à chaque position) et $J \in \mathbb{R}^{L \times L \times A \times A}$ les couplages de paires (épistasie). **ProfileMLP** est entraîné sur des cibles y qui proviennent de ce score complet (F et J), alors que ses features d'entrée ne contiennent que l'information mono-site. Le réseau ne peut donc capturer un effet de paire que si ses couches cachées combinent implicitement deux blocs one-hot actifs simultanément dans x — rien ne l'y oblige, mais rien ne l'en empêche non plus.

La question de ce document : comment retrouver $\hat{F}$ et $\hat{J}$ à partir d'un modèle $\mathcal{M}$ qui ne sait faire qu'une chose, $x \mapsto \mathcal{M}(x) \in \mathbb{R}$, sans jamais toucher à ses poids internes (`linear1.kernel`, `batchnorm1`, ...) ?

2 Idée générale : sonder le réseau plutôt que lire ses poids

On ne lit *jamais* les tenseurs de poids internes de **ProfileMLP** (`nnx.Linear.kernel`, etc.) : ces poids vivent dans l'espace des couches cachées, pas dans l'espace des 140 acides-aminoés×positions, et il n'y a pas de dictionnaire simple de l'un vers l'autre à travers deux couches de **BatchNorm** + **gelu** + **Dropout**. À la place, on traite $\mathcal{M}$ comme une boîte noire et on l'interroge à des entrées choisies exprès :

- $x_0 = \mathbf{0}$ — séquence de référence, aucun bloc actif ;
- $x_{p,a}$ — un seul bloc actif : la position p porte l'acide aminé a (*mutant simple*) ;
- $x_{i,a,j,b}$ — deux blocs actifs : position i porte a , position j porte b (*mutant double*).

C'est le même principe qu'un scan mutationnel en biologie expérimentale (deep mutational scanning) : on ne observe jamais les poids d'un système, seulement l'effet de mutations contrôlées sur sa sortie.

Hypothèse implicite. Cette méthode ne récupère un $\hat{F}, \hat{J}$ fidèle que si $\mathcal{M}$ se comporte, sur ces entrées éparpillées, comme une fonction additive-par-position plus une correction de paires — exactement la structure que l'entraînement sur des features mono-site *encourage*, sans la garantir. Un $x_{p,a}$ ou $x_{i,a,j,b}$ n'a par ailleurs que 1 ou 2 blocs actifs sur $L = 7$, alors que **ProfileMLP** n'a jamais vu, à l'entraînement, que des séquences réelles avec exactement L blocs actifs. On extrapole donc volontairement hors de la distribution d'entraînement.

3 Récupérer F : balayage mono-mutant

Pour chaque position p et chaque acide aminé a , on construit le mutant simple $x_{p,a}$ et on lit le score obtenu, comparé à la référence :

$$\hat{F}(a, p) = T \cdot (\mathcal{M}(x_{p,a}) - \mathcal{M}(x_0)),$$

où T est la température de la piste concernée (`protocol.T_viab` ou `protocol.T_sel`) — le même facteur d'échelle que celui utilisé par `RegressionV1.fit_weights_potts` pour que $\hat{F}$ soit directement comparable à `protocol.F_viab / protocol.F_sel`.

Pourquoi ça marche. Si $\mathcal{M}$ se décompose, sur ces entrées éparses, comme $\mathcal{M}(x) = c + \sum_p f_p(x_p)$ avec $f_p(0) = 0$ (ce que le score de référence $c = \mathcal{M}(x_0)$ fixe naturellement), alors $\mathcal{M}(x_{p,a}) = c + f_p(a)$, donc $f_p(a) = \mathcal{M}(x_{p,a}) - c$ — exactement la formule ci-dessus, à la constante T près. 140 = $L \cdot A$ passes avant suffisent (plus la référence).

4 Récupérer J : balayage double-mutant

Même idée, un cran plus loin. Pour chaque paire de positions $i < j$ et chaque paire d'acides aminés (a, b) , on construit le mutant double $x_{i,a,j,b}$ (les deux blocs (i, a) et (j, b) actifs à la fois, tout le reste à zéro) et on calcule le **résidu** au-delà de ce que les deux effets simples (p, a) et (j, b) expliquent déjà :

$$\Delta_{ij}(a, b) = \mathcal{M}(x_{i,a,j,b}) - \mathcal{M}(x_{i,a}) - \mathcal{M}(x_{j,b}) + \mathcal{M}(x_0),$$

$$\hat{J}(i, j, a, b) = \hat{J}(j, i, b, a) = T \cdot \Delta_{ij}(a, b).$$

C'est la mesure d'épistasie classique par *cycle mutationnel* (double-mutant cycle) : score du double mutant, moins ce que chaque mutation simple apporte déjà, moins la référence. Les deux entrées $\hat{J}(i, j, a, b)$ et $\hat{J}(j, i, b, a)$ sont remplies pour respecter la même convention de symétrie que `sequence_classesV1.build_J`, même si `protocol.compute_score` ne relit ensuite que la moitié $i < j$ du tenseur.

Pourquoi ça marche. En prolongeant l'hypothèse d'additivité de la section précédente avec un terme de paire, $\mathcal{M}(x) = c + \sum_p f_p(x_p) + \sum_{i < j} g_{ij}(x_i, x_j)$ avec $g_{ij}(0, \cdot) = g_{ij}(\cdot, 0) = 0$:

$$\mathcal{M}(x_{i,a,j,b}) = c + f_i(a) + f_j(b) + g_{ij}(a, b) \implies g_{ij}(a, b) = \mathcal{M}(x_{i,a,j,b}) - c - f_i(a) - f_j(b),$$

et en remplaçant $f_i(a) = \mathcal{M}(x_{i,a}) - c$, $f_j(b) = \mathcal{M}(x_{j,b}) - c$, on retombe exactement sur $\Delta_{ij}(a, b)$ ci-dessus. **Cette identité est exacte**, pas approximative, dès lors que $\mathcal{M}$ a effectivement cette forme additive + paires sur les entrées éparses testées — vérifié numériquement sur un modèle jouet synthétique reproduisant exactement cette structure (erreur de reconstruction $< 10^{-7}$, à la précision flottante près). Pour un vrai `ProfileMLP` entraîné, la question n'est pas si la formule est juste, mais si le réseau a effectivement appris une fonction de cette forme.

5 Le code (`extract_effective_FJ`)

```

1 def extract_effective_FJ(model, T, L=L, A=A):
2     base = np.zeros((1, L * A), dtype=np.float32)
3
4     # 1. Mutants simples : L*A entrees, une par (position, acide amine)
5     single_mutants = np.tile(base, (L * A, 1))
6     for pos in range(L):
7         for aa in range(A):
8             single_mutants[pos * A + aa, pos * A + aa] = 1.0
9
10    # 2. Mutants doubles : un bloc de A*A lignes par paire (i, j), i < j
11    pairs = [(i, j) for i in range(L) for j in range(i + 1, L)]
12    n_pairs = len(pairs)
13    double_mutants = np.tile(base, (n_pairs * A * A, 1))
14    for k, (i, j) in enumerate(pairs):
15        for aa_i in range(A):
16            for aa_j in range(A):
17                idx = k * (A * A) + aa_i * A + aa_j
18                double_mutants[idx, i * A + aa_i] = 1.0

```

```

19         double_mutants[idx, j * A + aa_j] = 1.0
20
21     # 3. Un seul forward pass pour tout le batch (reference + simples +
22         doubles)
23     x = jnp.asarray(np.concatenate([base, single_mutants, double_mutants
24         ], axis=0))
25     scores = np.asarray(model(x, train=False))
26
27     base_score      = scores[0]
28     single_scores   = scores[1 : 1 + L * A]
29     double_scores   = scores[1 + L * A :]
30
31     # 4. F : delta simple, mis a l'echelle par T
32     F_flat = (single_scores - base_score).reshape(L, A)
33     F_hat  = (F_flat * T).T.astype(np.float32) # (A, L)
34
35     # 5. J : residu du cycle mutationnel, mis a l'echelle par T, rempli
36         symetriquement
37     J_hat = np.zeros((L, L, A, A), dtype=np.float32)
38     for k, (i, j) in enumerate(pairs):
39         for aa_i in range(A):
40             for aa_j in range(A):
41                 idx      = k * (A * A) + aa_i * A + aa_j
42                 delta_ij = (double_scores[idx] - base_score
43                     - (single_scores[i * A + aa_i] - base_score)
44                     - (single_scores[j * A + aa_j] - base_score)
45                     )
46                 J_hat[i, j, aa_i, aa_j] = delta_ij * T
47                 J_hat[j, i, aa_j, aa_i] = delta_ij * T
48
49     return F_hat, J_hat
50
51 F_viab_hat, J_viab_hat = extract_effective_FJ(model_viab, protocol.
52     _T_viab)
53 F_sel_hat, J_sel_hat  = extract_effective_FJ(model_sel, protocol.
54     _T_sel)

```

Points d'implémentation à noter :

- Un seul `model(x, train=False)` pour toutes les entrées (référence + mutants simples + mutants doubles) : $1 + L \cdot A + \binom{L}{2} A^2$ lignes, soit $1 + 140 + 21 \times 400 = 8541$ pour $L = 7, A = 20$. Un seul passage en mode `train=False` désactive Dropout et fige BatchNorm en mode évaluation, donc le score est déterministe pour un même x .
- `scores = np.asarray(model(x, train=False))` rapatrie tout le tableau du GPU vers le CPU *une seule fois*, avant les boucles Python qui indexent `single_scores/double_scores` — sans ça, chaque itération de la boucle sur J (8400 itérations) déclencherait une synchronisation `device→host` individuelle.
- `F_hat` est transposé en (A, L) et `J_hat` construit en (L, L, A, A) pour matcher exactement la convention de `RegressionV1.fit_weights_potts` et de `sequence_classesV1.initialize_random_weight` — `F_hat` et `J_hat` sont directement comparables à `protocol.F_viab` et `protocol.J_viab` sans transformation supplémentaire.

6 Vérifier la récupération

`evaluate_profile_recovery` compare $\hat{F}, \hat{J}$ à la vérité terrain de deux façons complémentaires :

1. **Corrélation au niveau des poids** : `pearson` entre `protocol.F_viab.ravel()` et `F_viab_hat.ravel()` (et pareil pour J) — est-ce que chaque coefficient individuel est bien retrouvé ?
2. **Corrélation au niveau des scores** : on repasse $\hat{F}, \hat{J}$ dans `protocol.compute_score` (la même fonction que le simulateur utilise pour générer les vraies données) et on compare aux scores calculés avec les vrais `F_viab/J_viab`. Cette deuxième vue est plus indulgente aux ambiguïtés de symétrie/constante d'un cycle mutationnel : deux tenseurs J différents peuvent donner le même score total si leurs différences s'annulent systématiquement sur les séquences réelles.

```

1 v_hat = np.array(protocol.compute_score(F_viab_hat, J_viab_hat))
2 v_gt  = np.array(protocol.compute_score(protocol.F_viab, protocol.J_viab
3                                     ))
3 r_viab_scores = pearson(v_gt, v_hat)

```

7 Limites

- **Hors distribution.** Les entrées sondées (1 ou 2 blocs actifs sur $L = 7$) ne ressemblent à aucune séquence vue à l'entraînement (toujours L blocs actifs). `BatchNorm` en particulier a été calibré sur des statistiques de séquences complètes ; son comportement sur un x quasi-nul est une extrapolation, pas une interpolation.
- **Ambiguïté de symétrie/constante.** Le cycle mutationnel identifie J à une transformation additive près qui laisse les scores de doubles mutants inchangés (typiquement : déplacer une constante entre F et J). $\hat{J}$ peut donc différer de J_{vrai} terme à terme tout en produisant les mêmes scores — d'où l'intérêt de comparer aussi au niveau des scores (section précédente), pas seulement coefficient par coefficient.
- **Coût combinatoire.** Le nombre de mutants doubles croît en $\binom{L}{2}A^2$: gérable pour $L = 7, A = 20$ (8400 lignes), mais pas extensible tel quel à des séquences beaucoup plus longues sans sous-échantillonnage des paires.

8 Résumé

`extract_effective_FJ` ne lit jamais les poids internes de `ProfileMLP` : il sonde le réseau à des entrées choisies (référence nulle, mutants simples, mutants doubles) et lit les *différences de score* qui en résultent. Un delta simple $\mathcal{M}(x_{p,a}) - \mathcal{M}(x_0)$ recouvre l'effet de profil $\hat{F}(a, p)$; un résidu de cycle mutationnel $\mathcal{M}(x_{i,a,j,b}) - \mathcal{M}(x_{i,a}) - \mathcal{M}(x_{j,b}) + \mathcal{M}(x_0)$ recouvre le couplage de paire $\hat{J}(i, j, a, b)$. Les deux sont mis à l'échelle par la température T de la piste (viabilité ou sélectivité) et rangés dans les mêmes conventions de forme que `RegressionV1`, pour être directement comparables — coefficient par coefficient ou via `protocol.compute_score` — à la vérité terrain F, J du simulateur.